CPS721: Assignment 2

Due: October 15, 2026, 11:59pm
Total Marks: 125 (worth 4% of course mark)
In groups of no more than 3

Late Policy: Late submissions are penalized by 10% per day.

Clarifications and Questions: Please use the discussion forum on the D2L site to ask questions. These will be monitored regularly. A Frequently Asked Questions (FAQ) page will also be created. Only ask questions by email if it is specific to your submission. Any questions asked with less than 24 hours left to the deadline will not be answered.

PROLOG Instructions. Your code must run on ECLiPSe 7.1. If you cannot install this on your own machine, test it on the lab machines or the department moon servers.

You are NOT allowed to use ";" (disjunction), "!" (cut), "->" (if-then), `setof`, `bagof`, `findall`, or any other language features and library predicates that are not covered in class. Similarly, you should only use "=" and "is" for equality, and "not" for negation. Do NOT use "=:=", "\=", or other such commands. You will be penalized if you use these commands/features. You are only allowed to use ";" to get additional responses when interacting with PROLOG from the command line (this is equivalent to using the "More" button in the ECLiPSe GUI).

Note, this also means you are NOT allowed to use the built-in `member` or `append` predicates. However, you can reimplement these. If you do so, make sure to give it a name that doesn't clash with the built-in predicates (*ie.* `myMember`).

Academic Integrity and Collaboration Policy: Any submission you make must be the work of your group and your group alone. You are also not allowed to post course materials (including assignments, slides, etc.) anywhere on the web, and you are also prohibited from using generative AI systems or online "help" websites for completing assignments.

Submission Instructions: You should submit ONE zip file called `assignment2.zip` on D2L. It should contain the following files:

```
assignment2_submission_info.txt
q1_list_equality.pdf              q2_polynomial_calculation.pl
q3_contains_any.pl                q4_round_trip.pl
q5_find_stays.pl                  q6_cheapest_hotel.pl
q7_tree_as_terms.pl
```

All these files have been given to you and you should fill them out using the format described. Ensure your names appear in all files — especially `assignment2_submission_info.txt` — and your answers appear in the file sections as requested. Do NOT edit, delete, or add any lines that begin with "`%%%%% SECTION:`". These are used by the auto-grader to locate the relevant code. You will lose marks if you edit such lines.

Your submission should not include any other files than those above. You will be penalized for using compression formats other than `.zip`. Submissions by email will not be accepted.

You may submit your solution multiple times. Doing so will simply overwrite your previous submission. We will therefore always mark the last submission your group makes.

1 List Equality [15 marks]

For each of the following pairs of lists, state which can be made identical and which cannot. You must also provide a short proof as to why. This means you should convert the lists to the same style (*ie.* the '`|`' based representation or the standard ',' based representation), and use that to explain how they can be made identical, element by element, or why they can't be made identical. For example, if given the pairs `[X, Y]` and `[a | [b]]`, you could say that these match with `X = a` and `Y = b` since the second list can be written as `[a, b]` (or equivalently the first list can be written as `[X | [Y]]`). Any answers that do not contain explanations for why the lists do or do not match will lose marks.

a. `[truck, T, boat | [T, car, plane] ]` **and** `[N | [boat, M | [P | Q ] ] ]`

b. `[a, [M | [b,  c] ],  d ]` **and** `[a, [b, [b, c] ] | T]`

c. `[X | [ X | [ [ X | [ [X] ] ] ] ] ]` **and** `[b | M]`

d. `[U | [W |  [U] ] ]` **and** `[the, quick, brown, fox, W]`

e. `[Fox, [[in], socks], [on], box, on | [[knox]] ]` and
`[[The, cat], [[in], The], Hat | [Comes | Back] ]`

Each of these pairs is worth 3 marks. You should submit your answers in a pdf file called `q1_list_equality.pdf`. The names, emails, and student IDs of your group members should appear at the top of this PDF file.

Note, you are permitted to use ECLiPSe to check the answers to these questions, but you need to provide the proof itself.

2 Polynomial Calculation [10 marks]

Write a program to calculate the value of a polynomial like $5 - 7x + 3x^2$, when given the value of x. For example, if $x = 2$, then the calculated result of this polynomial is $5 - 7 \cdot 2 + 3 \cdot 2^2 = 3$. The program will be called `polyCalc(Coeffs, VarValue, Result)` where `Coeffs` is the list of coefficients of the terms (starting from the 0th power), `VarValue` is the value of x, and `Result` is the outcome of the computation. For example, the polynomial above is represented by the coefficient list `[5, -7, 3]` and so `polyCalc([5, -7, 3], 2, X)` should succeed with `X = 3`.

Note that `X^Y` computes `X` to the exponent `Y`. In addition, your program should return 0 if it is given an empty list. We will not test with enormous numbers or non-integer values for x to avoid overflow or floating point issues. You can assume that whenever your program is called, `Coeffs` and `VarValue` will not be variables.

Submit your program, including any helper predicates, in the `q2_rules` section of the file `q2_polynomial_calculation.pl`.

3 Contains Any [10 marks]

Write a program, `containsAny(List, ToSearchFor, Indices)`, that finds the list of `Indices` of all the elements of `List` for which there is an element that appears in `ToSearchFor`. The following shows examples of the expected behaviour of this program:

- `containsAny([a, b, 1, a, c, p, 2], [b, 2, q], X])` will succeed with `X = [1, 6])` because the element at index 1 of `List` (ie. `b`) is in the list `ToSearchFor`, and the element at index 6 of `List` (ie. 2) is also in ToSearchFor.
- `containsAny([a, c, c, c, b, a], [a, b, c, d], X)` will succeed with `X = [0, 1, 2, 3, 4, 5]` since every element in `List` appears in `ToSearchFor`.

Note that we assume that lists start with index 0. You can also assume that `Items` and `Contains` are always given as inputs (*ie.* we will never test with them set as variables).

Submit your program, including any helper predicates, in the `q3_rules` section of the file `q3_contains_any.pl`.

4 Finding a Round Trip Using Lists [30 marks]

For this question, you will explore writing a program for finding a round trip, just like on Assignment 1. The difference is that you will now be using lists both to avoid exploring paths that have loops in them, and to store trip information. Note, for this question, you will use KBs that use the same predicates as those defined in Assignment 1:

- `flight(City1, City2, Date, Price)`
- `hasRoom(Name, City, Date, Price)`
- `ticket(Name, City, Date, Price)`

In particular, you will write a program for the following predicate:

```
roundTrip(City, Start, End, NumFlights, FlightDates, Cities, Hotels, Cost)
```

As in Assignment 1, this query will hold if there are `NumFlights` flights starting in `City` and ending back in `City`. The first flight must be on day `Start` and the last flight must be on day `End`. For every day from `StartDate` to the day before `EndDate`, there must be an available hotel. The total cost of flights and hotels will be `Cost`. In addition, `FlightDates` will be a list of the dates that the flights are on, `Cities` is a list of the cities that those flights arrive in, `Hotels` is a list of the hotels the trip will stay in, and the round trip visits the same city no more than once.

For example, suppose our knowledge base contains a $200 flight from Toronto to Montreal on the 15th, a $150 flight from Montreal to Quebec City on the 17th, and a $250 flight from Quebec City to Toronto on the 20th. Moreover, assume there is a hotel (The Hilton) in Montreal that costs $195 on each of the 15th and 16th, and a hotel in Quebec City (The Sheraton) that costs $125 on each of the 17th, 18th, and 19th. Then the following holds:

```
roundTrip(toronto, 15, 20, 3, FlightDates, Cities, Hotels, Cost).
```

with `Flights = [15, 17, 20]`, `Cities = [montreal, quebec_city, toronto]`, `Hotels = [the_hilton, the_sheraton]`, and `Cost = 1365` since $200 + 2 \cdot 195 + 150 + 3 \cdot 125 + 250 = 1365$.

Assignment 1 defines other requirements on this predicate. In addition, note the following:

- No city should be visited more than once. This includes the starting city, which should not be returned to until the very last day.
- The flight dates, cities, and hotels, must all appear in chronological order in their respective lists.
- Duplicate hotel names in multiple cities are permitted. For example, the trip may involve a stay at The Hilton in both Montreal and Quebec City.
- You can always assume `Start`, `End`, and `NumFlights` will be given as input (*ie.* not as variables). For full marks, you should be able to handle any combination of the other arguments being variables. However, we suggest you first get the program working with the starting `City` being set as a constant before trying to get it working when set as a variable.
- If we call "More", your program should allow us to find all possible combinations of flight dates, cities, hotels, and costs that satisfies the given constraints query.

Submit your program, including any helper predicates, in the `q4_rules` section of the file `q4_round_trip.pl`. YOUR TEST KB SHOULD NOT APPEAR IN THIS FILE. A section for importing your KB from another file has been given. You do NOT need to submit the KB you use for testing.

Importantly, this file should compile and run on its own. If you reuse any helpers from the previous assignment (or other questions in this assignment), they must be included in the `q4_rules` section of the file `q4_find_stays.pl`.

5 List of Stay Terms [20 marks]

For this question, we will create another predicate that could be used as part of our automated trip planning system. The task here is to find available hotel rooms given a list of cities to visit and the dates for the travel between them. In this case, the assumption is that the user has already manually found transportation for the days in question (*ie.* flights, train, car, etc.).

For the purpose of this program, we will use the following term:

```
stay(HotelName, City, CheckInDate, CheckOutDate, TotalCost)
```

This indicates that it is possible to stay at the hotel named `HotelName` in `City` for the given `TotalCost`, starting from `CheckInDate` and leaving on `CheckOutDate`.

Your task is to write a program that takes in a list of dates and cities, and finds a list of `stay` terms which provides a list of hotel stays for the whole trip. The program will be defined as follows:

- `findStays(Dates, Cities, Stays)` where `Dates` is a list of travel days, `Cities` are the cities being visited in chronological order, and `Stays` are the list of hotel stays in those cities.

Note that `FlightDates` and `Cities` will have the same format as the previous question. For example, consider a slight variant of the example given in the previous question. In particular, assume the user has already booked train tickets from Toronto to Montreal on the 15th, then Montreal to Quebec City on the 17th, and finally Quebec City to Toronto on the 20th. In addition, assume there is a hotel (The Hilton) in Montreal that costs $195 on each of the 15th and 16th, and a hotel in Quebec City (The Sheraton) that costs $125 on each of the 17th, 18th, and 19th. Now consider the following query.

```
findStays([15, 17, 20], [montreal, quebec_city, toronto], X).
```

In our example, this will hold with

```
X = [ stay(the_hilton, montreal, 15, 17, 390),
          stay(the_sheraton, quebec_city, 17, 20, 375)]
```

In addition, note the following:

- You should assume the KB defining the available hotel rooms will have statements using the `hasRoom` predicate defined exactly as done in Assignment 1 and in the previous question.
- We will assume the user does not want to change hotels during their time in any one city, so a stay should last the entire time from one travel day to the next.
- Since we are already assuming the user has transportation organized, your program should NOT check if there is available flight on the dates given, since the user may be using a different form of transportation.
- The list of stays should appear in chronological order. You can assume the given list of cities are in chronological order of when they are visited.
- Your program should fail if `Dates` contains less than 2 elements, or the given list of cities has a different length than the given list of dates. Similarly, if the given dates are not in order or two dates are the same, your program should fail since it is not a valid query.
- You can always assume that `Dates` and `Cities` will be given as input when your program is called (*ie.* `findStays` will never be queried with `Dates` and `Cities` as variables).
- If we call "More", your program should allow us to find all possible combinations of stays that satisfies the given query.

Submit your program, including any helper predicates, in the `q5_rules` section of the file `q5_find_stays.pl`. YOUR TEST KB SHOULD NOT APPEAR IN THIS FILE. A section for importing your KB from another file has been given. You do NOT need to submit the KB you use for testing.

Importantly, this file should compile and run on its own. If you reuse any helpers from the previous assignment (or other questions in this assignment), they must be included in the `q5_rules` section of the file `q5_find_stays.pl`.

6 The Cheapest Stay [20 marks]

In the previous question, you created a program that will find a list of stays for the given set of cities. In this question, you will write a program to find the cheapest stay in such a given list of stays. That is, you will write a program as follows:

- `cheapestHotel(Stays, Cheapest)` will hold if `Cheapest` is the cheapest stay in the given list of `Stays`.

For example, suppose you have a list of stays as follows:

```
Stays = [ stay(the_hilton, montreal, 15, 17, 390),
          stay(the_sheraton, quebec_city, 17, 20, 375),
          stay(the_marriott, toronto, 20, 25, 1000),
          stay(holiday_inn, hamilton, 25, 29, 400)]
```

then the query `cheapestHotel(Stays, X)` will succeed with
`X = stay(the_sheraton, quebec_city, 17, 20, 375)`. In addition, note the following:

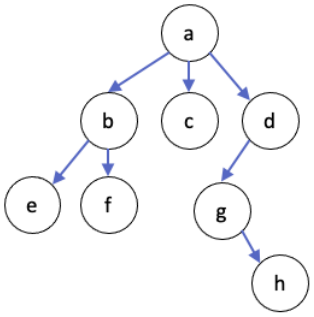
- If `Stays = []`, then `cheapestHotel(Stays, X)` should fail, since the notion of the cheapest stay in an empty list is undefined.
- You can assume that `Stays` will be given as input in any tests (*ie.* `cheapestHotel` will never be called with `Stays` set as a variable).
- For additional flexibility (*ie.* to allow your program to be used in more contexts), you should NOT assume the list of input stays are in chronological order, or are non-overlapping, or even that they are all for different cities.
- For simplicity, you can assume there will be no ties in the costs of the hotels in the given list of stays.

Submit your program, including any helper predicates, in the `q6_rules` section of the file `q6_cheapest_hotel.pl`. YOUR TEST KB SHOULD NOT APPEAR IN THIS FILE. A section for importing your KB from another file has been given. You do NOT need to submit the KB you use for testing.

Importantly, this file should compile and run on its own. If you reuse any helpers from the previous assignment (or other questions in this assignment), they must be included in the `q6_rules` section of the file `q6_cheapest_hotel.pl`.

7 Trees as Terms [20 marks]

Recall that a postorder traversal of a tree means that a node is visited after its children. For example, consider the following ternary tree, which is a tree where each node has at most 3 children (much like how a binary tree has at most two children at every node):



A postorder traversal would mean that the nodes are visited in the following order: e, f, b, c, h, g, d, a. In this question, you will write a program that takes in a ternary tree and generates the list which corresponds to the postorder traversal of the tree. To represent a ternary tree, we will use the following term `tree3(Name, Left, Middle, Right)`. Here, `Name` is the name of the node, and `Left`, `Middle`, and `Right` represent the three branches of the tree. The values of each these will be terms: either a `tree3` term or `none` indicating there is no branch below.

For this question, you will use the term `next(Head, Tail)` to represent a Prolog list `[Head | Tail]`. Here, we will use `nil` to represent the empty list `[]`. For example,
`next(7, next(1, next(5, next(0, next(9, nil)))))` represents the list `[7,1,5,0,9]`.

Your program for listing the tree elements as they would be in a postorder traversal should then be defined as follows:

`postorder(Tree, Order)` - where `Tree` is the given tree and `List` is the list of elements as they appear during the postorder traversal.

For example, if your program is called as `postorder(Tree, X)` given the tree above, then your program should succeed with

```
X = next(e, next(f, next(b, next(c, next(h, next(g, next(d, next(a, nil)))))))))
```

In addition, note the following:

- We will not test your program with `Tree` as a variable.
- If `Tree = none`, then your program should succeed with the list `nil`.

You should write your program in `q7_rules` section of the file `q7_tree_as_terms.pl`. You may add helper predicates as you see fit. The tree above, and a few other trees, have already been included in that file for testing purposes. You may want to test on additional trees to ensure your program works in all cases. Note that we have used an extra predicate in the file to make testing easier. See the file for more information. Make sure you do not put any such trees in the `q7_rules` section of the given file. You are NOT allowed to use standard Prolog list notation in your solution program.